	Questions covered from chapter 1 and 2
Q1.	An engineer needs to find and visualize the minimum value of the function using Python. $$f(x) = x^2 - 4x + 5$$ a. Discuss how different Python modules can work together to solve this scientific computing problem. Identify: b. Which module can be used for numerical optimization to find the minimum of the function? c. Which module can be used to generate values of x for plotting? d. Which module can be used to visualize the function and its minimum point? Explain how these modules work together in solving the problem
Q2.	An engineer needs to calculate the area under the curve $f(x)=x^2$ between 0 and 5 numerically. Which Python module can be used, and which type of operation from that module is suitable?
Q3.	A research laboratory receives a large numerical problem involving vector calculations, graphical representation, symbolic differentiation, numerical integration, and 3D visualization. Propose a suitable Python module for each task and explain why using specialized modules is advantageous.
Q4.	Explain the advantages of using Thonny for beginners, distinguish Pip from a development environment, and explain why changing the Python version may affect the availability of previously installed Python modules.
Q5.	Explain Spyder as a development environment for scientific and engineering computing. Discuss its relationship with the Anaconda distribution and distinguish between Pip and a development environment, including the role of Pip and the Python Package Index (PyPI).
Q6.	Design a small scientific-computing application in Python that combines at least two modules introduced in Chapter 1. State the problem, identify the modules, describe the input and output, and explain how the modules work together.
Q7.	A triangular arrangement of sensors is installed in a monitoring system. Row i contains i sensors, numbered sequentially from 1 to i. Write a Python program using nested loops to display the sensor-number arrangement for N rows.
Q8.	A security system generates the following access-code pattern for a given positive integer N. Write a Python program to generate the pattern using nested loops. For N = 5 1 12 123 1234 12345
Q9.	A data-processing system receives a sequence of integers representing identifiers. It must locate the first element that occurs more than once, where “first” means the

element whose second occurrence appears earliest while scanning from left to right.
For example:
Input: 7 3 5 3 9 5 2
Output: 3
Write a Python program that:
1. Reads a list of integers.
2. Searches for the first repeated element using loops.
3. Prints the repeated element and its second-occurrence position.
4. Prints "No repeated element" if every element is unique.
Do not use set(), count(), or any built-in frequency/counting function.

Q10. An electricity distribution company wants to develop a Python program to calculate the monthly electricity bill of a customer based on the number of units consumed. The company follows the following tariff structure:

Units Consumed	**Energy Charge per Unit**
First 100 units	₹5 per unit
Next 100 units (101–200)	₹7 per unit
Above 200 units	₹10 per unit

In addition to the energy charge, a **5% tax** is charged on the calculated energy charge.
Write a modular Python program using **user-defined functions** to calculate and display the customer's electricity bill.

Q11. A simple student-record system stores marks in a list. Write a function linear_search(marks, target) that searches for target using a loop and returns the index of its first occurrence. If the target is not present, the function must return -1.

The main program should:

- read a list of marks
- read the target mark
- call the function
- report whether the mark was found and its position.

For example:

Marks: 72 81 65 90 81 77

Target: 81

Output:

Mark found at index 1

The search must terminate immediately when the first matching value is found. Do

	not use the in operator, index(), or any other built-in searching operation.
Q12.	A university maintains student records: students = [("S101", "Amit", [78, 85, 91, 67]), ("S102", "Neha", [92, 89, 95, 90]), ("S103", "Ravi", [56, 64, 71, 60]), ("S104", "Priya", [85, 90, 84, 88])] Write a Python program to: • Convert the records into a dictionary where the student ID is the key and the student's name and marks are stored as values. • Calculate the total and average marks of every student and identify the student with the highest average. • Using filter() and a lambda function, create a collection containing students whose average marks are greater than 80. • Using sorted() and a lambda function, sort the students according to their average marks in descending order. • Create a set containing the IDs of students who scored more than 85 in at least one subject.
Q13.	An organization stores employee information as: employees = [("E101", "Alice", "IT", 65000), ("E102", "Bob", "HR", 52000), ("E103", "Charlie", "IT", 78000), ("E104", "David", "Finance", 72000), ("E105", "Eva", "IT", 85000)] Write a Python program to: a. Create a dictionary where each department maps to a set of employee IDs belonging to that department.

	b. Find the highest-paid employee and calculate the average salary using appropriate built-in functions. c. Use filter() and lambda to obtain employees earning more than ₹70,000. d. Sort employees by salary in descending order using a lambda function. e. Find employees who belong to the IT department and earn more than ₹70,000 by combining set membership and dictionary/list processing.

Q14.	A temperature-monitoring system stores measurement:
	
	sensor_data = {
	
	"S01": [23.5, 25.2, 27.1, 29.8],
	
	"S02": [21.4, 24.8, 26.3, 25.7],
	
	"S03": [30.2, 31.5, 29.9, 32.1]
	
	}
	
	Write a Python program to:
	
	• Calculate the average temperature recorded by each sensor and store the result in a new dictionary.
	
	• Find the sensor having the highest average temperature.
	
	• Use map() and a lambda function to convert all temperatures from Celsius to Fahrenheit.
	
	• Create a set containing all distinct temperature values greater than 30°C.
	
	• Sort the sensors according to their average temperature using sorted() and a lambda function.

Q15	Develop an object-oriented banking application. Create a base class BankAccount containing account number, account holder, and balance. • Implement methods for deposit(), withdraw(), and displaying account details. Ensure that invalid transactions are rejected. • Encapsulate the balance so that it cannot be directly modified from outside the class. • Create a subclass SavingsAccount with an interest rate and override an

	appropriate method to calculate/add interest. • Create multiple account objects and store them in a list of objects. Use a lambda function with sorted() to sort accounts according to their balance. • Create a dictionary mapping account types to the set of account numbers belonging to each type.

Q16.	A library maintains book records:
	
	books = {
	
	"B101": {"title": "Python Programming", "author": "A", "copies": 4},
	
	"B102": {"title": "Data Structures", "author": "B", "copies": 2},
	
	"B103": {"title": "Operating Systems", "author": "C", "copies": 5},
	
	"B104": {"title": "Database Systems", "author": "D", "copies": 1}
	
	}
	
	Write a Python program to:
	
	• Display books having fewer than 3 available copies.
	
	• Add a new book, update the number of copies of an existing book, and remove a book using dictionary operations.
	
	• Create a set of all authors and determine the number of unique authors.
	
	• Create a list containing book IDs sorted by the number of available copies using a lambda function.
	
	• Create a tuple containing the titles of all books currently having at least 3 copies.

Q17.	Develop an object-oriented vehicle rental system. Create a base class Vehicle having vehicle number, model, rental rate, and availability status. • Create subclasses Car, Bike, and Truck, each with an appropriate additional attribute and customized rental-cost calculation. • Demonstrate inheritance, method overriding, encapsulation, and polymorphism by calculating rental costs through a common interface. • Store the vehicle objects in a list and use filter() with a lambda function to find all currently available vehicles.

	• Use sorted() and a lambda function to arrange available vehicles according to rental cost. • Maintain a dictionary where vehicle type is the key and the corresponding value is a set of vehicle numbers.

Q18.	A college wants to analyze student attendance and academic performance. Design an object oriented system using a class Student.
	
	Each student should contain:
	
	• Student ID
	
	• Name
	
	• Attendance record
	
	• Marks
	
	For example:
	
	attendance = {
	
	"Python": 26,
	
	"DBMS": 23,
	
	"OS": 28
	
	}
	
	marks = (82, 76, 91)
	
	Write a Python program to:
	
	• Implement methods to calculate the student's average attendance and average marks.
	
	• Create multiple Student objects and store them in a list. Use filter() and lambda to
	
	find students whose attendance is at least 75% in every subject.
	
	• Use sorted() with a lambda function to rank students according to their average marks.
	
	• Create a dictionary mapping each subject to a set of students who attended at least 75% of its classes.
	
	• Create a tuple containing the IDs of students who satisfy both conditions: average marks ≥ 80 and average attendance ≥ 75%.